

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE

SISTEMAS COMPUTACIONALES



TÓPICOS ESPECIALES II

PROYECTO 1: DESARROLLO ASISTIDO POR IA CON HERRAMIENTAS DE ESCRITORIO

EVIDENCIAS DEL PROYECTO – SISTEMA DE RESERVAS WEB

Grupo: 1GS242

Profe: Regis Rivera

Estudiantes:

He, Kelvin 8-999-1950

Barrera, Roy 8-1022-2121

Mosquera, Einer 8-924-1880

28/10/2026

ÍNDICE

1. Información general del proyecto
2. Análisis del proyecto existente
3. Historias de usuario
4. Repositorio Git, issues, commits, ramas y Pull Requests
5. Pruebas automatizadas
6. Evidencia del uso de IA (AI Development Log)
7. Ejemplos de respuestas incorrectas de la IA
8. Cambios que se rechazaron de la IA
9. Decisiones tomadas por humanos
10. Presentación
11. Checklist de cumplimiento

1. Información general del proyecto

Este documento reúne las evidencias del Proyecto 1 (Desarrollo asistido por IA con herramientas de escritorio) del curso Tópicos Especiales II, a cargo del Prof. Regis Rivera. El foco de la evaluación es el proceso: cómo se usó la IA, cómo se validó, qué se corrigió y qué se rechazó.

Campo	Detalle
Repositorio base elegido	CitaFácil – https://github.com/gianca675/citafacil (licencia MIT)
Repositorio del grupo	https://github.com/kelvinhe04/citafacil (público). Base clonada y subida a repositorio propio con su historial.
Stack (front-end / back-end / BD)	HTML5 + CSS3 + JavaScript (sin framework) / Python 3 + FastAPI + SQLAlchemy + Pydantic / SQLite (local) · PostgreSQL (remoto)
Herramientas y modelos de IA	Claude Code (agente de escritorio) con modelos en la nube: Claude Sonnet 5.5 (Kelvin y Einer) y Claude Opus 5.5 (Roy). Playwright para verificar en navegador y GitHub CLI para issues y PR.
Marco de trabajo	Feature Sprint con IA sobre un sistema de reservas web existente
Fecha límite de entrega	Miércoles 28/10/2026, 6:00 PM (asignación en MS Teams)

1.1 Equipo y rotación de roles

Cada integrante rota entre Developer, Reviewer y QA a lo largo de las historias de usuario.

Historia	Developer	Reviewer	QA
HU-01 Reserva con datos guardados	Kelvin	Roy	Einer
HU-02 Calendario dinámico	Roy	Einer	Kelvin
HU-03 Gráficos de previsión de demanda	Einer	Kelvin	Roy
HU-04 (propia)	Kelvin	Roy	Einer
HU-05 (propia)	Roy	Einer	Kelvin

Integrantes: Kelvin He (8-999-1950), Roy Barrera (8-1022-2121) y Einer Mosquera (8-924-1880).

2. Análisis del proyecto existente

Antes de pedirle a la IA que modifique código se analizó el repositorio elegido. Esta sección documenta lo que se encontró y cómo se usó la IA como apoyo para entender el proyecto.

2.1 Arquitectura y estructura de carpetas

Aplicación full-stack de tres capas: frontend estático (HTML5, CSS3 y JavaScript sin framework: index.html y admin.html), API REST en FastAPI (routers público, admin y scores en backend/app/routers) y acceso a datos con SQLAlchemy sobre SQLite (local) o PostgreSQL (Render). Módulos principales: models.py, schemas.py (Pydantic), crud.py, availability.py (horarios libres y anti doble-booking), notifications.py y reminders.py (WhatsApp con Twilio, modo demo sin credenciales), auth.py (cabecera X-Admin-Token) y config.py (variables de entorno). Línea base: 6 pruebas pytest.

2.2 Modelo de datos

Dos tablas: services (id, name, duration_min, price, active) y appointments (id, customer_name, customer_phone en formato E.164, service_id, start_at, end_at, status = pending/confirmed/cancelled/done, notes, reminder_sent, created_at). No existen usuarios, cuentas, pagos ni inventario: el cliente se identifica solo por nombre y celular. No hay migraciones: create_all únicamente crea tablas nuevas.

2.3 Flujos relevantes para las historias

La reserva pública tiene tres pasos (servicio, día y hora, datos): GET /api/services, GET /api/availability y POST /api/appointments, que valida el celular chileno, comprueba que el horario esté libre (409 si choca) y envía el WhatsApp de confirmación. El panel de administración, protegido por token, lista y filtra citas, cambia su estado, gestiona servicios, muestra estadísticas y dispara recordatorios; APScheduler los envía cada 15 minutos.

2.4 Hallazgos y limitaciones

1) No hay formulario de pago ni cuentas: HU-01 se adaptó a «datos guardados» (nombre y celular). 2) No hay migraciones: una columna nueva rompería la base desplegada, por eso HU-04 usa un token firmado. 3) is_slot_free cuenta la cita propia como ocupada: se añadió exclude_id para poder reprogramar. 4) La API de producción por defecto del front apunta al despliegue del autor original (Render), que respondió error 500 durante las pruebas: todo se probó en local con ?api=. 5) Bugs previos hallados al probar: los errores 422 se mostraban como «[object Object]», tras un error se borraba la hora elegida y la fecha mínima usaba UTC. 6) El histórico estaba vacío (la app no permite reservar en el pasado y seed.py solo creaba citas futuras), lo que impedía ver la previsión.



3. Historias de usuario

Las tres historias obligatorias vienen del enunciado; las dos adicionales son de elección del grupo (formato Como… quiero… para…). Para cada una se documenta desarrollo, prueba y validación.

ID	Historia de usuario	Issue	Rama	PR	Estado
HU-01	Como usuario recurrente, quiero reservar directamente usando mis datos guardados, para no tener que rellenar el formulario de pago en cada compra.	#1	feature/hu-01-datos-guardados	#6	Fusionado. Revisó Roy (aprobó). QA: Einer (sin registro).
HU-02	Como cliente, quiero ver un calendario dinámico con los días disponibles y ocupados, para elegir mi fecha rápidamente sin intentos fallidos.	#2	feature/hu-02-calendario	#12	Fusionado. QA: Kelvin (aprobó). Revisión de Einer: sin registro.
HU-03	Como administrador del sistema, quiero ver gráficos de previsión de la demanda basados en el histórico, para ajustar los precios y el inventario de forma inteligente.	#3 (duplicado: #9)	feature/hu-03-prevision-demanda	#10	Fusionado. Revisó Kelvin. QA: Roy (aprobó).
HU-04	Como cliente, quiero cancelar o reprogramar mi cita desde un enlace, para no tener que llamar al negocio. (propuesta del grupo)	#4	feature/hu-04-cancelar-reprogramar	#7	Fusionado. Revisó Roy (aprobó). QA: Einer (sin registro).
HU-05	Como administrador, quiero exportar las reservas a CSV, para llevar el control en una hoja de cálculo.	#5	feature/hu-05-exportar-csv	#13	Fusionado. Comentario de Einer. QA: Kelvin (aprobó).

3.1 HU-01: Reserva con datos guardados

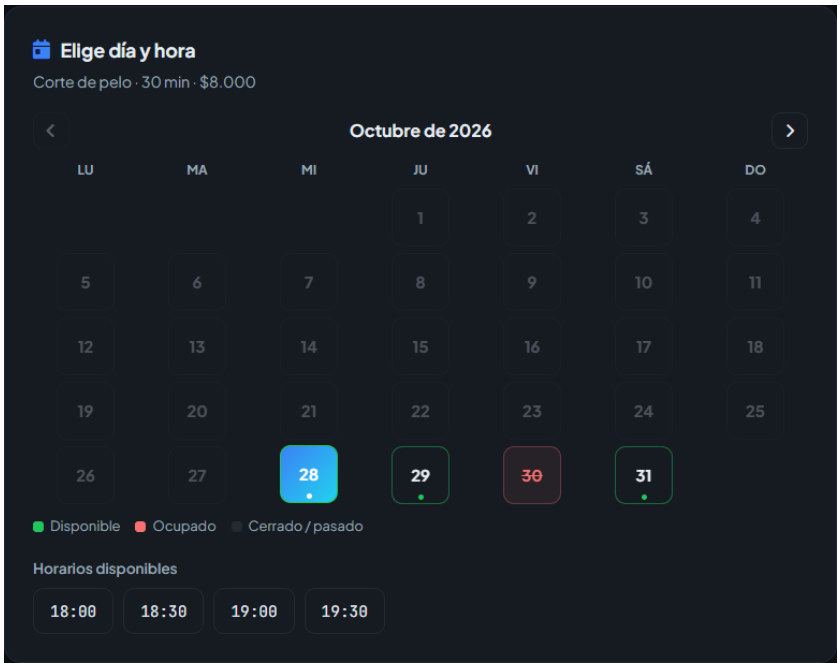
Resultado esperado	El cliente recurrente reserva con un clic usando el nombre y el celular que el navegador guardó de su reserva anterior; puede usar otros datos u olvidarlos.
Criterios de aceptación	1) Tras reservar se guardan nombre y celular (el celular normalizado que devuelve la API) en localStorage. 2) En la visita siguiente el paso 3 ofrece «Reservar con mis datos guardados». 3) «Usar otros datos» y «Olvidar mis datos» funcionan. 4) Los datos se siguen validando (celular E.164). 5) No hay endpoint de búsqueda por celular (privacidad). 6) Pruebas automatizadas nuevas.
Qué hizo la IA / qué se ajustó a mano	Claude Code (Sonnet 5.5, Kelvin) analizó index.html, schemas, crud y tests antes de programar y propuso guardar los datos solo en el navegador. Implementó book() compartida, saveCustomer/loadSaved y 3 pruebas. Decisiones 3, 4 y 5 del AI Log.
Cómo se probó y validó	pytest: 9 passed en la rama (6 + 3). Navegador (Playwright): cliente nuevo ve el formulario; cliente recurrente reserva en un clic y la cita queda en el servidor; «Olvidar» limpia; un nombre con HTML se muestra como texto. Revisión: Roy aprobó el PR #6.



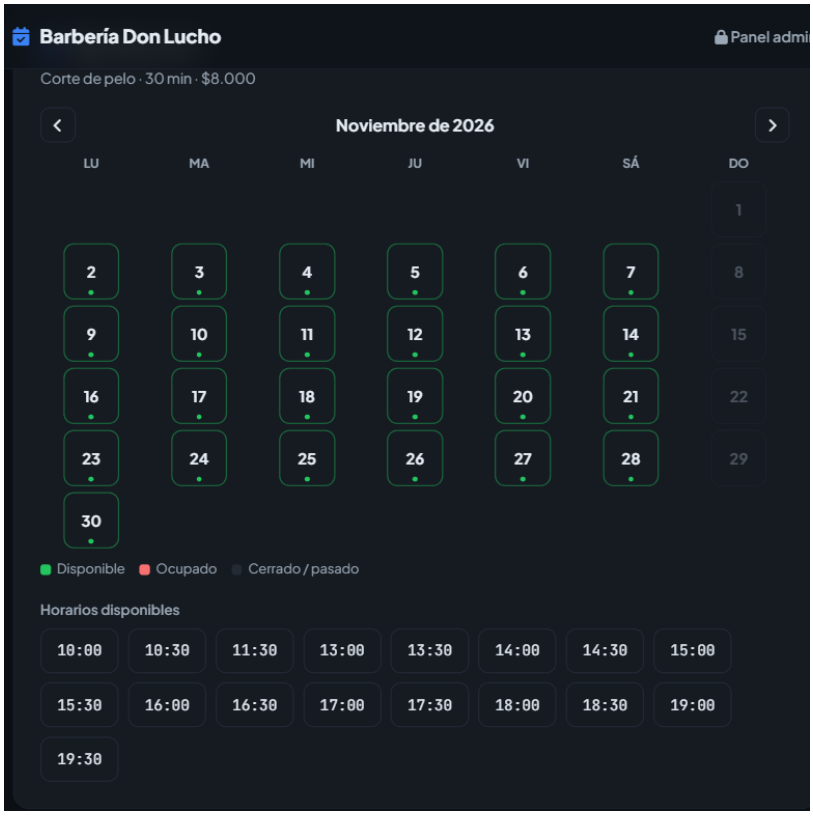
HU-01: paso 3 con «Reservar con mis datos guardados» (segunda visita).

3.2 HU-02: Calendario dinámico

Resultado esperado	El cliente ve un calendario mensual con los días disponibles, ocupados y cerrados/pasados del servicio elegido. Al tocar un día disponible ve sus horarios libres, sin intentos fallidos.
Criterios de aceptación	1) Endpoint GET /api/availability/month?service_id=&month=YYYY-MM que reutiliza availability.py. 2) Calendario con días disponibles (verde), ocupados (rojo, tachado) y cerrados/pasados (gris) distinguibles. 3) Elegir un día disponible muestra sus horarios libres. 4) Pruebas automatizadas nuevas pasando. 5) Captura y registro en docs/AI_LOG.md.
Qué hizo la IA / qué se ajustó a mano	Claude Code (Claude Opus 5.5) analizó availability.py, routers/public.py, index.html y las ramas abiertas antes de programar. Propuso reutilizar generate_slots día por día (así el calendario y la lista de horarios nunca se contradicen) e implementó el endpoint, el calendario y 14 pruebas. A mano se corrigieron 2 errores visuales de la IA (título «Octubre De 2026» por text-transform: capitalize, y celdas desalineadas con los encabezados) y se rechazó tocar los mismos imports que HU-04 porque provocaba conflicto de merge. También corrige el bug de la fecha mínima en UTC que HU-01 dejó anotado.
Cómo se probó y validó	pytest: 20 passed (6 originales + 14 nuevas: mes completo, domingos cerrados, conteo igual a /api/availability, día lleno, cita cancelada que no bloquea, días pasados, febrero bisiesto, formatos de mes inválidos → 422 y servicio inexistente o inactivo → 404). Pre-chequeo del Developer en navegador con Playwright: estados del mes, clic en día lleno, navegación de meses, cambio de servicio y vista móvil. Integración con todas las ramas abiertas: 62 passed. Validación manual de QA: Kelvin (PR #12).



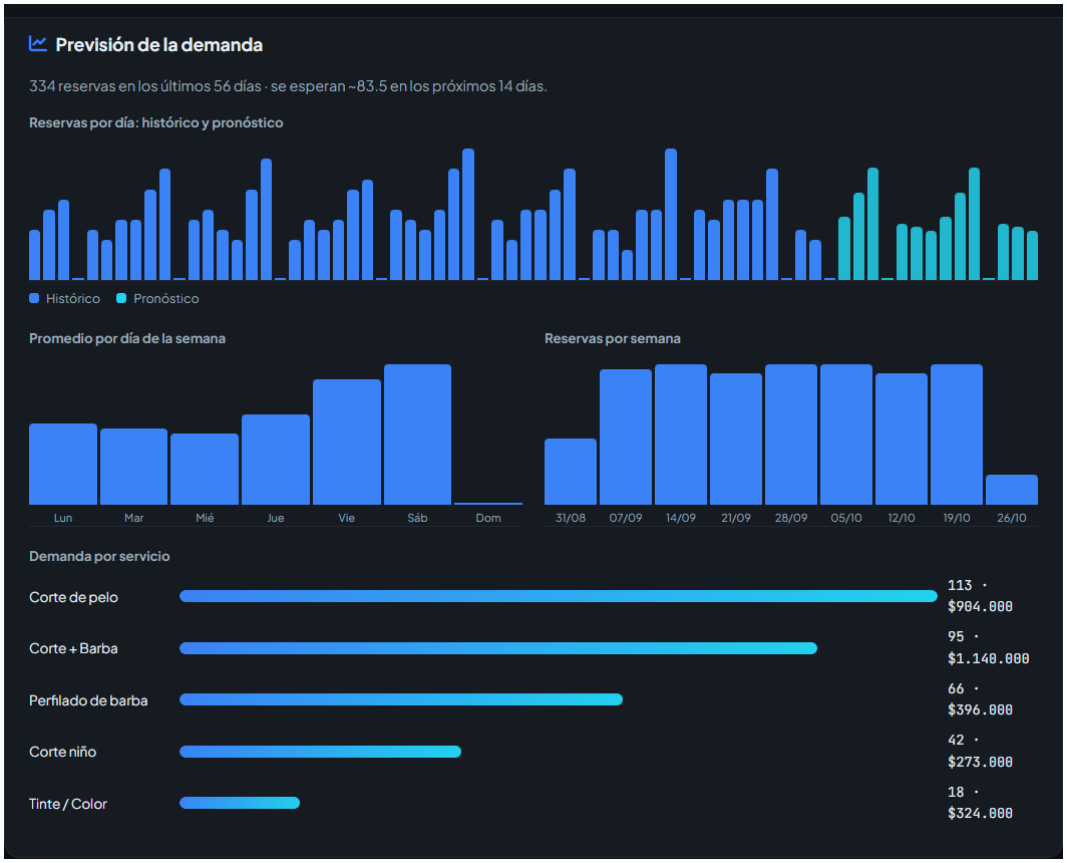
HU-02: calendario de octubre de 2026. Días 28, 29 y 31 disponibles, 30 ocupado y días pasados en gris; al elegir el 28 se muestran sus horarios libres.



HU-02 (QA de Kelvin, 28/10): calendario de noviembre con domingos cerrados; horarios coinciden con la API.

3.3 HU-03: Gráficos de previsión de demanda

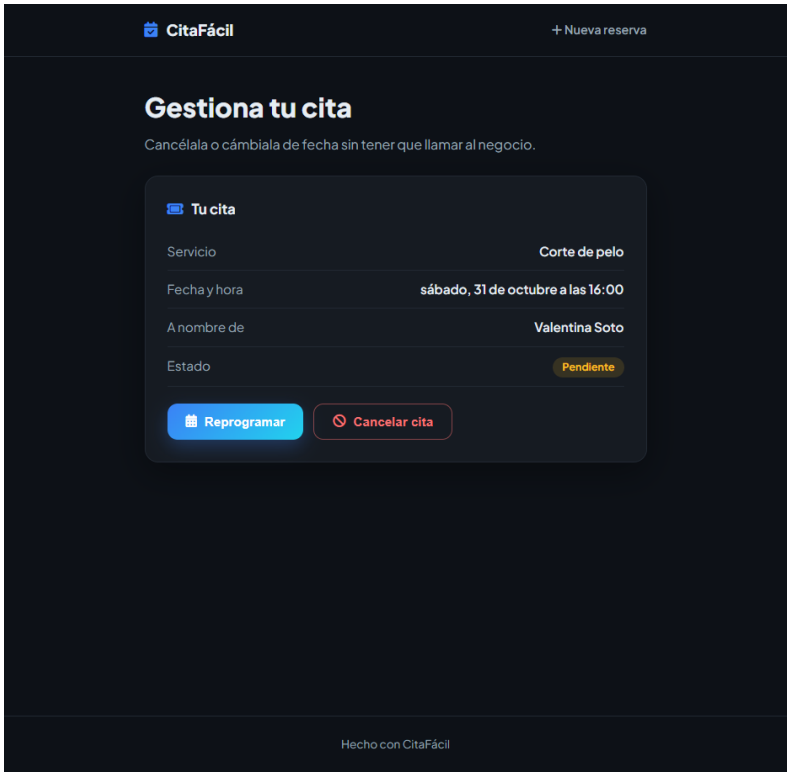
Resultado esperado	El administrador ve en el panel el histórico de reservas y una previsión de la demanda: gráficos por día (histórico + pronóstico), promedio por día de la semana, reservas por semana y demanda e ingresos por servicio.
Criterios de aceptación	1) GET /api/admin/forecast (history_days, horizon_days) protegido con X-Admin-Token. 2) Las citas canceladas no cuentan. 3) El pronóstico es el promedio del mismo día de la semana. 4) Gráficos sin librerías externas en admin.html. 5) 7 pruebas nuevas.
Qué hizo la IA / qué se ajustó a mano	Claude Code (Sonnet 5.5, Einer) implementó forecast.py, el endpoint, los gráficos y 7 pruebas. En la revisión, Kelvin verificó el cálculo con 92 citas de patrón conocido (coincidió) y detectó que sin histórico los gráficos salían en cero: se resolvió con el PR #11 (python seed.py --historico). Quedaron mejoras menores (semanas parciales, «ingresos estimados», una etiqueta). Una prueba esperaba 401 y el repo responde 422: se corrigió la prueba.
Cómo se probó y validó	QA (Roy, 28/10): pytest 13 passed en la rama del PR #10 (20 con el histórico de seed.py --historico del PR #11; 62 con todas las ramas integradas). API: sin token → 422, token incorrecto → 401, parámetros fuera de rango → 422, límites válidos → 200. Datos: 334 reservas = suma diaria = suma semanal = suma por servicio; el pronóstico es el promedio por día de la semana (~83.5 reservas en 14 días). Navegador (escritorio y móvil): los 4 gráficos se cargan sin errores. Observaciones no bloqueantes en el PR #10: el día de hoy cuenta como día completo, semanas parciales, etiqueta partida en dos líneas y barras muy finas en móvil.



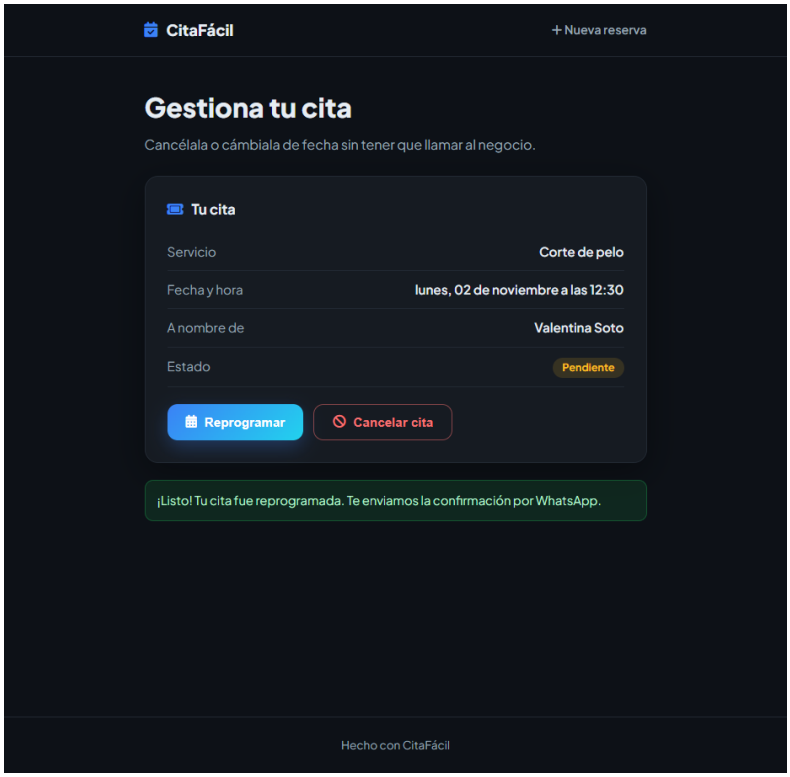
HU-03 (QA): previsión de la demanda en el panel admin con el histórico de ejemplo (56 días + 14 de pronóstico).

3.4 HU-04: Cancelar o reprogramar desde un enlace

Resultado esperado	El cliente cancela o reprograma su cita desde un enlace firmado, sin llamar al negocio.
Criterios de aceptación	1) Al reservar se devuelve un token y aparece «Gestionar mi cita». 2) manage.html permite ver la cita, reprogramarla (horarios libres sin bloquear la cita propia) y cancelarla. 3) Token HMAC sin cambios de esquema; un enlace falso responde 404. 4) Solo hasta 2 horas antes. 5) Al reprogramar se reinicia el recordatorio y se avisa por WhatsApp. 6) 15 pruebas nuevas.
Qué hizo la IA / qué se ajustó a mano	Claude Code (Sonnet 5.5, Kelvin) analizó el backend y detectó: no hay migraciones, is_slot_free cuenta la cita propia y reminder_sent debe reiniciarse. Implementó manage.py, el router /api/manage, manage.html y 15 pruebas. Dos errores de la IA corregidos (sección 7).
Cómo se probó y validó	pytest: 21 passed en la rama. Navegador: reservar, abrir el enlace, reprogramar (el servidor queda con la nueva fecha y el horario viejo se libera), cancelar y enlace falso. Revisión: Roy aprobó el PR #7 con observaciones no bloqueantes.



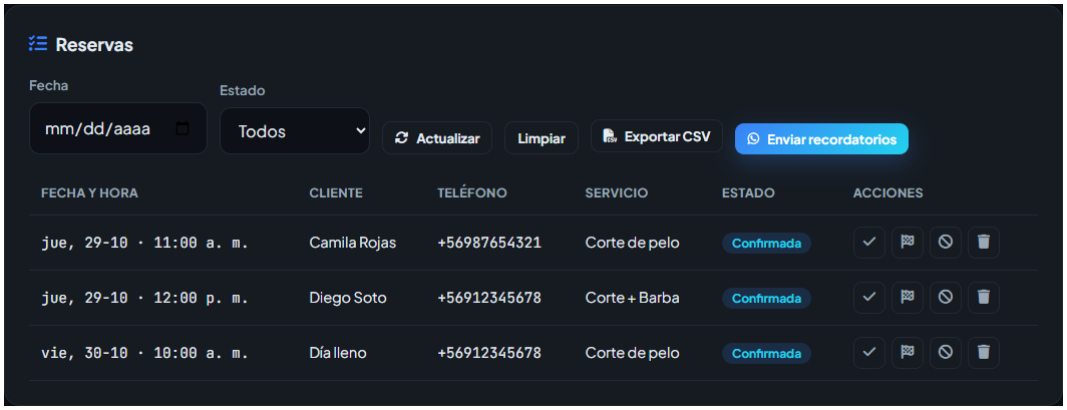
HU-04: página «Gestiona tu cita» abierta desde el enlace.



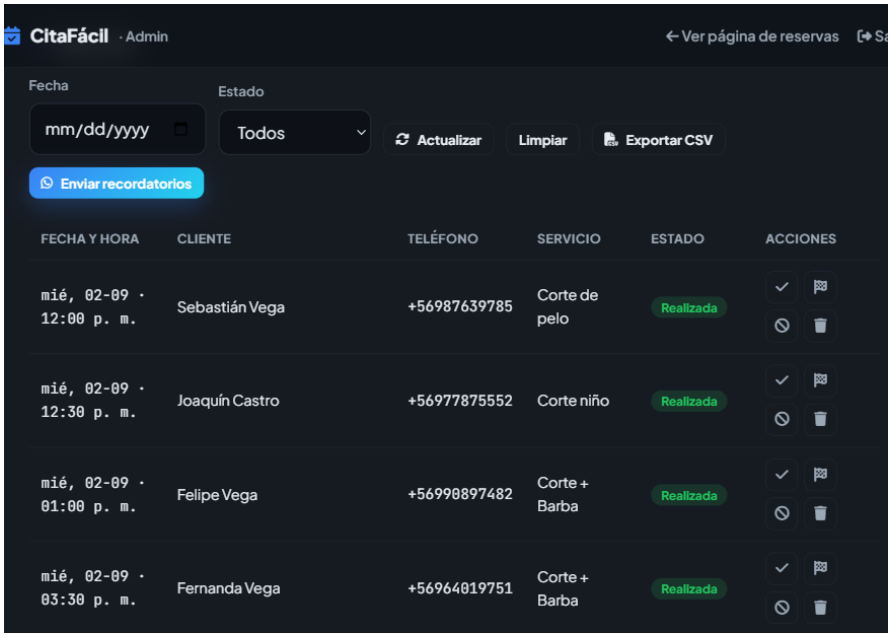
HU-04: cita reprogramada (el servidor quedó con la nueva fecha).

3.5 HU-05: Exportar reservas a CSV

Resultado esperado	El administrador descarga desde el panel un archivo CSV con las reservas, con los mismos filtros de fecha y estado que muestra la tabla, y lo abre en Excel o Google Sheets con tildes y eñes correctas.
Criterios de aceptación	1) Endpoint admin GET /api/admin/appointments/export con filtros por estado y rango de fechas, protegido con X-Admin-Token. 2) Botón «Exportar CSV» en admin.html. 3) Pruebas automatizadas nuevas pasando. 4) Captura y registro en docs/AI_LOG.md.
Qué hizo la IA / qué se ajustó a mano	Claude Code (Claude Opus 5.5) analizó crud.py, routers/admin.py y admin.html; propuso ampliar list_appointments con date_from/date_to sin romper el filtro del panel y un módulo export.py aparte. Implementó el CSV con BOM UTF-8, estados en español, neutralización de fórmulas (= + - @) en el texto que escribe el cliente y la descarga con fetch + Blob (el endpoint exige el token). Quedan para confirmar con el Reviewer: separador coma, el botón exporta lo que muestra la tabla y «precio» es el precio actual del servicio (la cita no lo guarda).
Cómo se probó y validó	pytest: 16 passed (6 originales + 10 nuevas: token, cabeceras de descarga y BOM, CSV vacío, contenido con tildes, orden por fecha, filtro por estado, rango inclusivo, neutralización de fórmulas, filtros inválidos → 422 y regresión del filtro del panel). Pre-chequeo en navegador con Playwright: el CSV tiene las mismas filas que la tabla, con y sin filtro. Integración con todas las ramas abiertas: 62 passed. Validación manual de QA: Kelvin (PR #13).



HU-05: botón «Exportar CSV» en el panel de reservas; descarga lo que muestra la tabla con los filtros actuales.



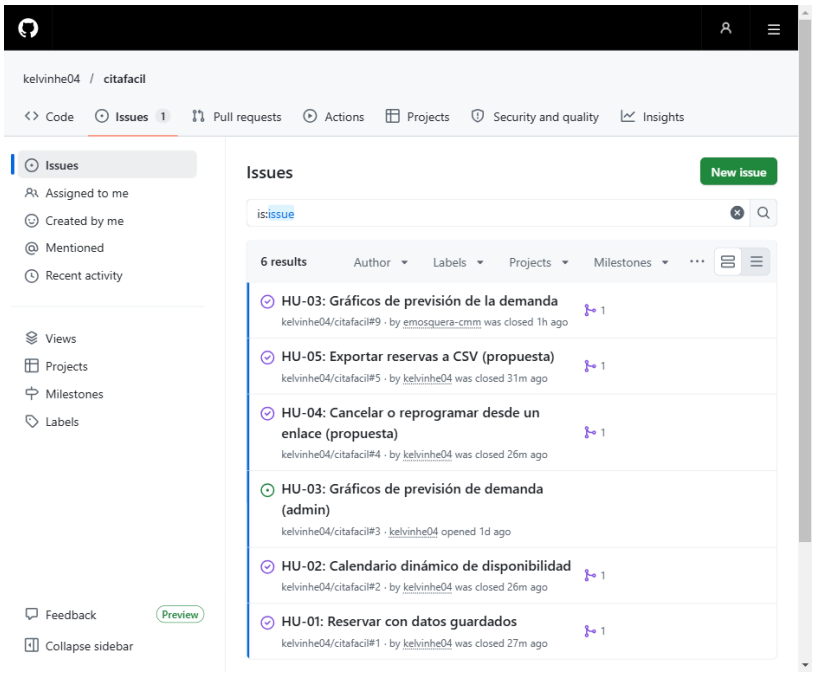
HU-05 (QA de Kelvin, 28/10): botón «Exportar CSV» y filtros en el panel.

4. Repositorio Git, issues, commits, ramas y Pull Requests

4.1 Repositorio

Repositorio del grupo: https://github.com/kelvinhe04/citafacil (público). Se clonó gianca675/citafacil (licencia MIT, se conserva su crédito y su historial) y se subió a un repositorio propio. En total: 5 issues de historias de usuario y 7 Pull Requests, todos fusionados en main.

4.2 Issues e historias de usuario

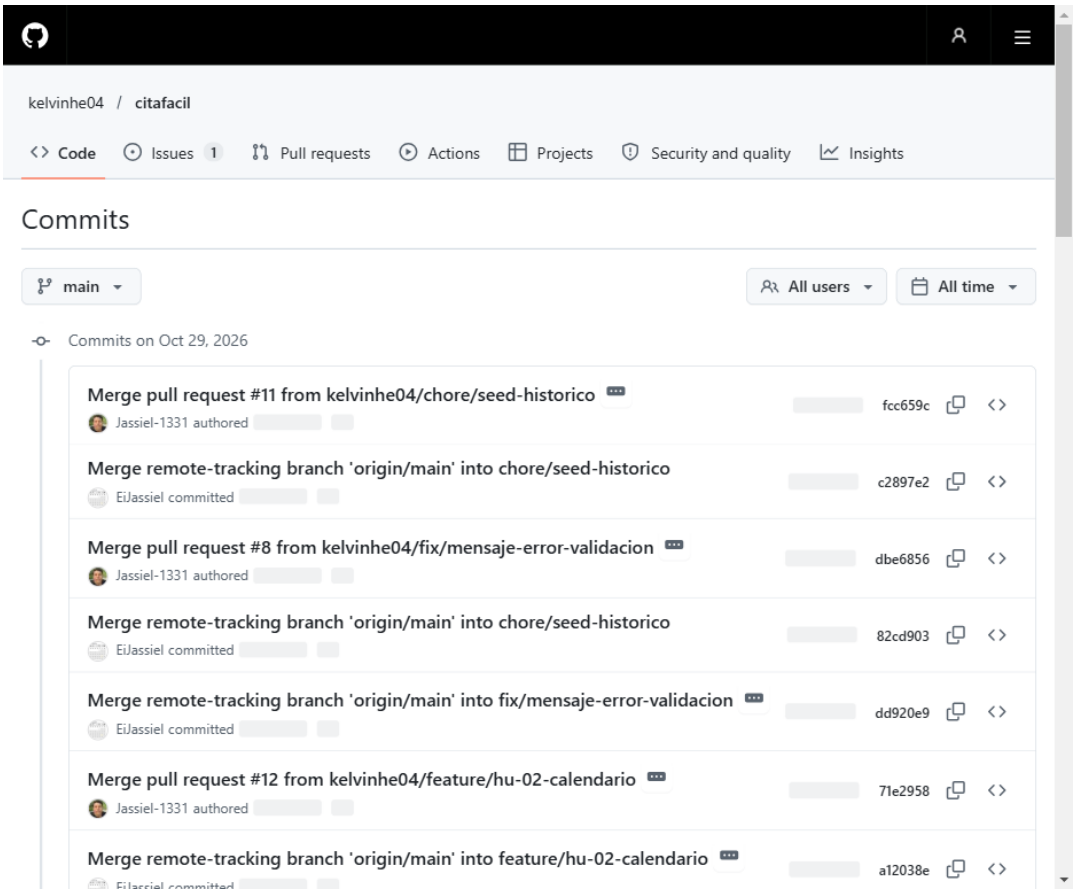


Issues del repositorio: una historia de usuario por issue.

4.3 Commits identificables

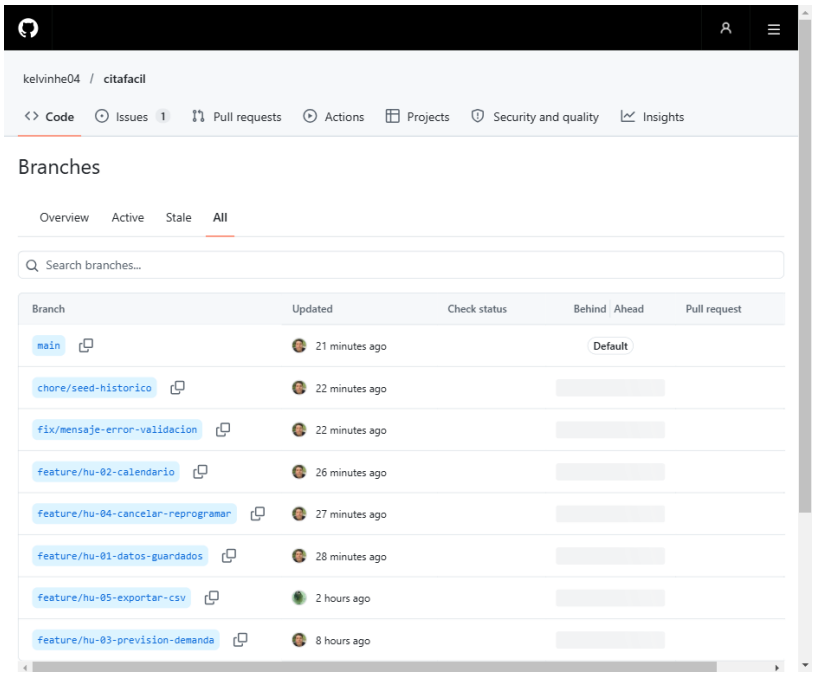
Commit	Autor	Mensaje	Historia
a11e351	Kelvin He	FIX: conservar la hora elegida si la reserva falla por un error que no es 409	FIX
65bb522	Kelvin He	FIX: muestra el mensaje de validación en vez de "[object Object]"	FIX
a392380	Kelvin He	HU-01: evidencia (captura) y registro en el AI Log	HU-01
4b9c93c	Kelvin He	HU-01: pruebas de reserva con datos guardados	HU-01
32c44fb	Kelvin He	HU-01: reserva en un clic con los datos guardados del cliente	HU-01
d40f3f9	Roy Barrera (Roy-x24)	HU-02: registra en el AI Log el uso de IA de la historia	HU-02
09f554c	Roy Barrera (Roy-x24)	HU-02: agrega capturas del calendario como evidencia	HU-02

Commit	Autor	Mensaje	Historia
19d7ea1	Roy Barrera (Roy-x24)	HU-02: reemplaza el selector de fecha por un calendario de disponibilidad	HU-02
f28c422	Roy Barrera (Roy-x24)	HU-03: agrega capturas del QA de la previsión de la demanda	HU-03
397aac0	Einer Mosquera	HU-03: agrega previsión de la demanda en el panel admin	HU-03
b9fa665	Kelvin He	HU-04: manage.html muestra mensajes de validación legibles (apiError)	HU-04
69f73c4	Kelvin He	HU-04: evidencia (capturas) y registro en el AI Log	HU-04
9085003	Kelvin He	HU-04: página manage.html y enlace "Gestionar mi cita" tras reservar	HU-04
fa3e6b8	Roy Barrera (Roy-x24)	HU-05: registra en el AI Log el uso de IA de la historia	HU-05
d273718	Roy Barrera (Roy-x24)	HU-05: agrega captura del panel con la exportación como evidencia	HU-05
4406419	Roy Barrera (Roy-x24)	HU-05: agrega botón Exportar CSV en el panel admin	HU-05
97a0f0c	Kelvin He	SEED: documenta python seed.py --historico en README y AGENTS.md	SEED
d15f263	Kelvin He	SEED: pruebas del histórico de ejemplo (7 casos)	SEED
f4c7548	Kelvin He	SEED: agrega histórico de ejemplo (python seed.py --historico)	SEED



4.4 Uso de ramas

Convención usada: feature/hu-0X-descripcion (una rama por historia), fix/… para correcciones y chore/… para tareas; main recibe todo por Pull Request.

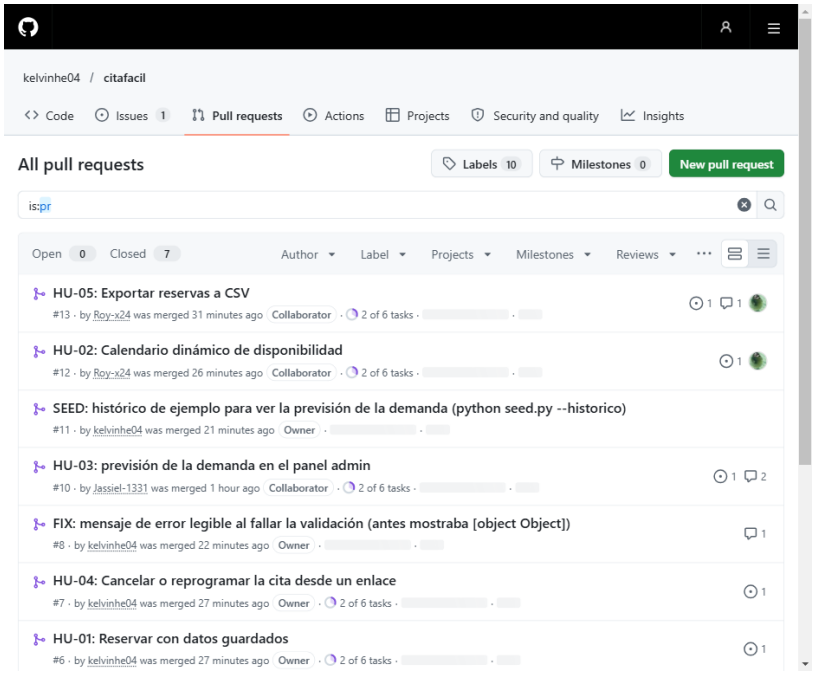


Ramas del repositorio: una por historia más el fix y el seed.

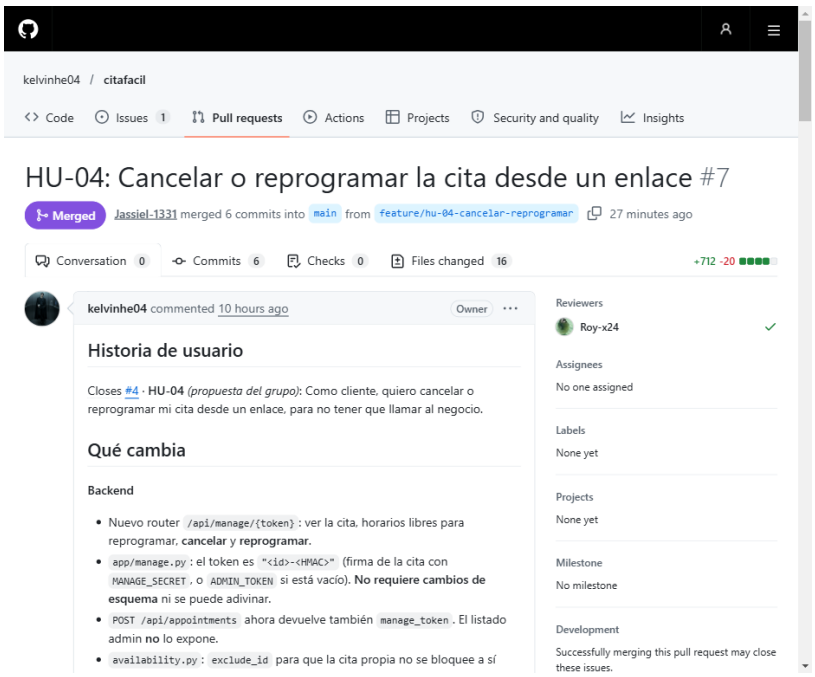
4.5 Pull Requests y revisión

PR	Título	Autor	Revisor	Resultado
#6	HU-01: Reservar con datos guardados	Kelvin	Roy	Aprobado por Roy; fusionado.
#7	HU-04: Cancelar o reprogramar la cita desde un enlace	Kelvin	Roy	Aprobado por Roy con observaciones no bloqueantes; fusionado.
#8	FIX: mensaje de error legible (antes «[object Object]»)	Kelvin	—	Fusionado sin revisión formal.
#10	HU-03: previsión de la demanda en el panel admin	Einer	Kelvin	Revisión con comentarios; QA de Roy aprobado; fusionado.
#11	SEED: histórico de ejemplo (seed.py --historico)	Kelvin	—	Fusionado sin revisión formal (Roy lo usó para el QA de HU-03).
#12	HU-02: Calendario dinámico de disponibilidad	Roy	Einer	Fusionado; revisión de Einer sin registro; QA de Kelvin aprobado.
#13	HU-05: Exportar reservas a CSV	Roy	Einer	Fusionado con un comentario de Einer; QA de Kelvin aprobado.

PR	Título	Autor	Revisor	Resultado



Los 7 Pull Requests, todos fusionados.



PR #7: revisión de Roy (Reviewer) marcada como aprobada.

5. Pruebas automatizadas

Historia	Tipo de prueba	Herramienta	N.º tests	Resultado
HU-01	Unitaria + integración (API)	pytest + FastAPI TestClient	3	9/9 en la rama
HU-02	Unitaria + integración (API)	pytest + FastAPI TestClient	14	20/20 en la rama
HU-03	Unitaria + integración (API)	pytest + FastAPI TestClient	7	13/13 en la rama
HU-04	Unitaria + integración (API)	pytest + FastAPI TestClient	15	21/21 en la rama
HU-05	Unitaria + integración (API)	pytest + FastAPI TestClient	10	16/16 en la rama
Seed	Unitaria (histórico de ejemplo)	pytest	7	13/13 en la rama
TOTAL en main	6 de la línea base + 56 nuevas	pytest	62	62 passed

Comando para ejecutar las pruebas: cd backend && python -m pytest -q

```
Salida real de pytest sobre main (28/10/2026):
test_booking.py: 9 passed
test_calendar.py: 14 passed
test_export_csv.py: 10 passed
test_forecast.py: 7 passed
test_manage.py: 15 passed
test_seed.py: 7 passed

62 passed, 2 warnings in 9.16s
```

6. Evidencia del uso de IA (AI Development Log)

Solo se registran las interacciones que afectaron significativamente el desarrollo; no es necesario registrar cada prompt.

Actividad	Herramienta IA	Qué se solicitó	Resultado	Decisión humana
Arquitectura (HU-04, Kelvin)	Claude Code (Claude Sonnet 5.5)	Analizar el backend antes de programar (modelos, crud, disponibilidad, recordatorios, config)	Detectó 3 restricciones: no hay migraciones (agregar una columna rompería la BD desplegada); is_slot_free cuenta la cita propia como ocupada; reminder_sent debe reiniciarse al reprogramar	Aceptada
Diseño (HU-04, Kelvin)	Claude Code (Claude Sonnet 5.5)	Proponer cómo identificar la cita sin cuentas de usuario	Token firmado (HMAC) sin cambios de esquema, en vez de una columna nueva en la BD	Aceptada; revisada y aprobada por Roy en el PR #7
Código (HU-04, Kelvin)	Claude Code (Claude Sonnet 5.5)	Backend: manage.py, router /api/manage/*, exclude_id en disponibilidad, mensajes de WhatsApp. Frontend: manage.html y enlace en la reserva confirmada	4 endpoints nuevos y página de autogestión (cancelar y reprogramar)	Aceptada tras corregir los 2 errores de la sección 2
Testing (HU-04, Kelvin)	Claude Code (Claude Sonnet 5.5)	Crear pruebas automatizadas de la historia	15 pruebas nuevas en test_manage.py; suite en 21/21	Aceptada
Testing (pre-chequeo del Developer) (HU-04, Kelvin)	Claude Code (Claude Sonnet 5.5) + Playwright	Probar en navegador: reservar, abrir el enlace, reprogramar, cancelar y enlace falso	Todo correcto (capturas en docs/evidencias/HU-04_*.png). La validación formal la hace Einer (QA)	Aceptada
Arquitectura (HU-01, Kelvin)	Claude Code (Claude Sonnet 5.5)	Analizar el repo (index.html, schemas, crud, tests) antes de modificar código	Detectó que el repo no tiene pago ni cuentas; que el backend ya normaliza el celular a +56… y lo devuelve; y que el flujo es de 3 pasos	Aceptada
Código (HU-01, Kelvin)	Claude Code (Claude Sonnet 5.5)	Reserva en un clic con datos guardados en el navegador, y "Olvidar mis datos"	Cambios en frontend/index.html (reserva rápida, localStorage, función book() compartida)	Aceptada; revisada y aprobada por Roy en el PR #6
Testing (HU-01, Kelvin)	Claude Code (Claude Sonnet 5.5)	Crear pruebas automatizadas de la historia	3 pruebas nuevas en test_booking.py; suite en 9/9	Aceptada
Testing (pre-chequeo del Developer) (HU-01, Kelvin)	Claude Code + Playwright	Probar en navegador: cliente nuevo, cliente recurrente, olvidar datos y nombre con HTML	Todo correcto (captura en docs/evidencias/HU-01_reserva-rapida.png). La validación formal la hace Einer (QA)	Aceptada

Actividad	Herramienta IA	Qué se solicitó	Resultado	Decisión humana
Implementación (HU-03, Einer)	Claude Code (Sonnet 5.5)	Endpoint de histórico/previsión de demanda, gráficos en admin.html y pruebas	forecast.py, GET /api/admin/forecast, gráficos de barras sin librerías, 7 pruebas nuevas (13 en total pasan)	Pendiente de revisión: probar a mano en el panel
Arquitectura (HU-02, Roy)	Claude Code (Claude Opus 5.5)	Analizar availability.py, routers/public.py, index.html y las ramas abiertas (HU-01, HU-03, HU-04, fix) antes de programar	Propuso reutilizar generate_slots día por día; detectó que HU-04 cambia la firma de generate_slots y el import de public.py (riesgo de conflicto)	Aceptada
Código (HU-02, Roy)	Claude Code (Claude Opus 5.5)	Endpoint GET /api/availability/month?service_id=&month=YYYY-MM con el estado de cada día	month_availability() en availability.py, esquema DayAvailabilityOut, validación del mes con regex (422) y 404 para servicio inactivo	Aceptada; pendiente de revisión por Einer
Código (HU-02, Roy)	Claude Code (Claude Opus 5.5)	Reemplazar el <input type="date"> por un calendario mensual	Calendario lunes-domingo con días disponibles (verde), ocupados (rojo, tachado) y cerrados/pasados (gris); solo los disponibles son clicables	Aceptada tras corregir los errores 3 y 4 de la sección 2
Testing (HU-02, Roy)	Claude Code (Claude Opus 5.5)	Crear pruebas automatizadas de la historia	test_calendar.py: 14 pruebas (incluye mes bisiesto, día lleno, cita cancelada que no bloquea y formatos de mes inválidos); suite en 20/20	Aceptada
Testing (pre-chequeo del Developer) (HU-02, Roy)	Claude Code (Claude Opus 5.5) + Playwright	Probar en navegador: estados del mes, clic en día lleno, navegación de meses, cambio de servicio y vista móvil	Todo correcto (capturas en docs/evidencias/HU-02_*.png). La validación formal la hace Kelvin (QA)	Aceptada
Arquitectura (HU-05, Roy)	Claude Code (Claude Opus 5.5)	Analizar crud.list_appointments, routers/admin.py y admin.html antes de programar	Propuso ampliar list_appointments con un rango date_from/date_to (sin romper el filtro date del panel) y un módulo export.py aparte	Aceptada
Código (HU-05, Roy)	Claude Code (Claude Opus 5.5)	Endpoint GET /api/admin/appointments/export y botón "Exportar CSV" en admin.html	CSV con BOM UTF-8, estados en español y neutralización de fórmulas; el botón descarga con fetch + Blob porque el endpoint exige X-Admin-Token	Aceptada; pendiente de revisión por Einer

Actividad	Herramienta IA	Qué se solicitó	Resultado	Decisión humana
Testing (HU-05, Roy)	Claude Code (Claude Opus 5.5)	Crear pruebas automatizadas de la historia	test_export_csv.py: 10 pruebas (auth, cabeceras, BOM, contenido, orden, filtros, rango inclusivo, fórmulas y filtros inválidos); suite en 16/16	Aceptada
Testing (pre-chequeo del Developer) (HU-05, Roy)	Claude Code (Claude Opus 5.5) + Playwright	Descargar el CSV desde el panel, sin filtros y filtrando por estado	Mismo número de filas en la tabla y en el CSV; BOM presente (captura en docs/evidencias/HU-05_exportar-csv.png). La validación formal la hace Kelvin (QA)	Aceptada
Evidencia de las conversaciones: cada Pull Request tiene la sección «Uso de IA» (qué se pidió y qué se aceptó, modificó o rechazó), y cada commit asistido lleva el trailer Co-Authored-By de la herramienta. El detalle fila por fila está en docs/AI_LOG.md.				

7. Ejemplos de respuestas incorrectas de la IA

N.º	Herramienta	Qué se pidió	Qué respondió mal	Cómo se detectó	Corrección
1	Claude Code (Claude Sonnet 5.5)	HU-04: Esquema ManageOut para la vista de la cita	Declaró can_change como campo obligatorio, pero el router lo rellena después de validar el objeto: toda petición a /api/manage/{token} fallaba con ValidationError	Las pruebas automatizadas: 5 fallaron en la primera ejecución	Se le dio valor por defecto (can_change: bool = False); suite en verde
2	Claude Code (Claude Sonnet 5.5)	HU-04: Script de parche para notifications.py (mensajes de WhatsApp)	Generó los \n de los textos como saltos de línea reales dentro de f-strings: código con error de sintaxis	Se verificó la sintaxis con ast.parse antes de ejecutar nada	Se reescribieron los 3 mensajes con \n escapados; sintaxis correcta
3	Claude Code (Claude Opus 5.5)	HU-02: Título del calendario ("octubre de 2026") con mayúscula inicial	Usó CSS text-transform: capitalize, que pone en mayúscula cada palabra: "Octubre De 2026"	Revisando la captura de Playwright	Se capitaliza solo la primera letra desde JavaScript
4	Claude Code (Claude Opus 5.5)	HU-02: Celdas cuadradas para los días	Limitó la altura (max-height) pero no el ancho: las celdas quedaban pegadas a la izquierda de su columna, desalineadas con los encabezados Lu…Do	Revisando la captura de Playwright	max-width + justify-self: center con aspect-ratio: 1/1
Evidencia: cada caso indica cómo se detectó y cómo se corrigió; los fallos de pytest y sus arreglos quedan en los commits y Pull Requests de cada historia (HU-04: PR #7, HU-02: PR #12).					

8. Cambios que se rechazaron de la IA

N.º	Cambio propuesto por la IA	Por qué se rechazó	Alternativa adoptada
2	Agregar from typing import Optional en availability.py y month_availability al import existente de public.py	Un merge de prueba con la rama de HU-04 dio conflicto: HU-04 agrega esas mismas líneas	date | None (válido con from __future__ import annotations) y el import nuevo en su propia línea; el merge con las demás ramas queda limpio

9. Decisiones tomadas por humanos

N.º	Decisión	Opciones evaluadas	Responsable	Justificación
1	Elegir CitaFácil como repositorio base	4 repos de la lista	Equipo	Python/FastAPI + SQLite sin framework de front; ya trae pruebas (6) y motor de disponibilidad
2	Adaptar HU-01 a "datos guardados" = nombre y celular (el repo no tiene pago)	Implementar un pago simulado	Equipo	Evitar inventar un flujo de pago ajeno al repo
3	Guardar nombre y celular solo en localStorage, sin endpoint de búsqueda por celular	Endpoint público que devuelva el nombre dado un celular	IA propuso; confirmada por Roy (Reviewer, PR #6)	Un endpoint así filtraría datos personales a cualquiera que conozca un número
4	Guardar el celular que devuelve la API (normalizado), no el tecleado	Guardar el texto tal como lo escribió el usuario	IA propuso; confirmada por Roy (Reviewer, PR #6)	Evita guardar formatos distintos del mismo número
5	No corregir en HU-01 el bug previo de la fecha mínima del calendario (usa fecha UTC: de noche en Panamá bloquea "hoy")	Corregirlo dentro de esta historia	IA propuso; resuelta en HU-02 (decisión 10), según Roy en el PR #6	Fuera de alcance; queda como mejora para otra historia (HU-02 toca el calendario)
6	Identificar la cita con un token firmado (HMAC) en vez de agregar una columna a la base de datos	Columna manage_token en appointments	IA propuso; confirmada por Roy (Reviewer, PR #7)	El proyecto no tiene migraciones: una columna nueva rompería la base ya desplegada
7	Solo se puede cancelar o reprogramar hasta 2 horas antes (MANAGE_MIN_HOURS_BEFORE)	Sin límite de anticipación	IA propuso; confirmada por Roy (Reviewer, PR #7)	Evita cancelaciones cuando ya no se puede reasignar el horario; es configurable
8	Al reprogramar se reinicia reminder_sent	Dejarlo como estaba	IA propuso; confirmada por Roy (Reviewer, PR #7)	Si no, el cliente no recibiría recordatorio de la nueva fecha
9	Cuatro estados por día: past, closed, full, available. "Hoy" sin horas restantes se muestra como ocupado	Solo disponible/no disponible; o un estado extra "pocos horarios"	Roy (Developer, con IA); pendiente de confirmar por Einer	Cumple el criterio (disponibles, ocupados y cerrados distinguibles) sin sobrecargar la vista; el número de horarios libres va en el tooltip
10	El calendario usa la fecha local del navegador y el backend decide qué días son pasados con la zona horaria del negocio	Mantener el min del input calculado con toISOString() (UTC)	Roy (Developer, con IA); pendiente de confirmar por Einer	Corrige el bug que HU-01 dejó anotado (decisión 5): de noche en Panamá la fecha UTC ya es "mañana" y bloqueaba "hoy"
11	No corregir en HU-02 el desborde horizontal de la página en móvil (418 px en 375 px)	Corregirlo aquí	Roy (Developer, con IA); pendiente de confirmar por Einer	Ya existía en main (lo causa el degradado .hero::before de 460 px), no lo introduce el calendario

N.º	Decisión	Opciones evaluadas	Responsable	Justificación
12	Separador coma + BOM UTF-8	Punto y coma (lo que espera Excel con configuración regional de Chile) o un parámetro sep	Roy (Developer, con IA); pendiente de confirmar por Einer	La coma es el estándar (RFC 4180) y la abren bien Google Sheets, LibreOffice y Excel en configuración de Panamá/EE. UU.; el BOM evita que Excel rompa tildes y eñes
13	El botón exporta exactamente lo que muestra la tabla (filtros de fecha y estado); el rango date_from/date_to queda disponible en la API	Agregar campos "Desde/Hasta" al panel	Roy (Developer, con IA); pendiente de confirmar por Einer	Mantiene el panel simple y lo que se descarga coincide con lo que se ve
14	Prefijar con ' el nombre, el servicio y las notas que empiecen con = + - @	Exportar el texto tal cual	Roy (Developer, con IA); pendiente de confirmar por Einer	Lo escribe el cliente desde la página pública: sin esto, un nombre como =HYPERLINK(...) se ejecutaría como fórmula al abrir el archivo. El celular no se toca porque el backend ya lo normaliza
15	La columna precio es el precio actual del servicio	Guardar el precio en cada cita	Roy (Developer, con IA); pendiente de confirmar por Einer	La tabla appointments no guarda el precio y el repo no tiene migraciones; queda como limitación conocida

10. Presentación

El presentador se elige al azar el día de la entrega, por lo que los tres integrantes deben dominar todo el proyecto y estar listos para recibir peticiones de cambio durante la presentación.

Elemento	Detalle
Material de apoyo	Presentación en diapositivas: docs/Presentacion_Proyecto1.pptx (y este documento de evidencias).
Demo preparada	App completa en local: cd backend && python seed.py --historico && python -m uvicorn app.main:app; en otra terminal cd frontend && python -m http.server 5500; abrir http://localhost:5500/index.html?api=http://localhost:8000. Orden: calendario (HU-02), reserva rápida (HU-01), gestionar cita (HU-04), panel admin con previsión (HU-03) y exportar CSV (HU-05).
Plan ante cambios en vivo	Los tres conocen todo el código. Un cambio pedido en vivo se hace con la IA de escritorio (Claude Code) en una rama aparte, se corre pytest antes de mostrarlo y se revierte con git si falla.

11. Checklist de cumplimiento

Lo que el enunciado exige demostrar (Restricciones del Proyecto 1) y la sección de este documento donde está la evidencia.

N.º	Requisito	Sección	Listo
1	Repositorio Git	4.1	☑
2	Issues o historias de usuario	3 y 4.2	☑
3	Commits identificables	4.3	☑
4	Uso de ramas	4.4	☑
5	Pull Request o revisión equivalente	4.5	☑ (3 de 7 PR con revisión formal)
6	Pruebas automatizadas	5	☑
7	Evidencia del uso de IA	6	☑
8	Ejemplos de respuestas incorrectas de la IA	7	☑
9	Cambios que rechazaron de la IA	8	☑ (1 caso)
10	Decisiones tomadas por humanos	9	☑